Recitation 8

Reichman University
Efi Arazi School of Computer Science

Recitation overview

- Strings – review
  - strlen, strcpy
- Step-by-step execution example of string functions
- Class exercise – strcat

Recitation overview

- Strings – review
  - strlen, strcpy
- Step-by-step execution example of string functions
- Class exercise – strcat

Strings in C

- Strings in C are implemented using **character arrays**
  - The array of an *n*-character string has *n*+1 entries, where last element is set to `'\0'` which has the value of zero in ASCII
  - The terminating `'\0'` is not a printable character, so it provides a way for encoding the string boundary

- String "abc" can be defined as regular array:

```
char s[] = {'a', 'b', 'c', '\0'};
```

- Using string literal to initialize array: (same consequence)

```
char s[] = "abc";
```

- Using string literal with free pointer:
  (string allocated in data segment with read-only access)

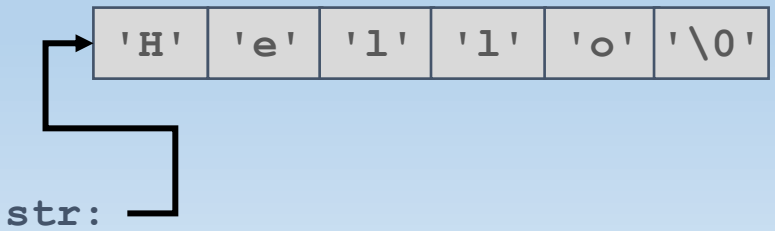
```
char *s = "abc";
```

Strings functions

A function for computing the length of a string:

```
int my_strlen(const char* str) {
   int len=0;
   while(str[len++])
     ;
   return len-1;
}
```

- The string is located in a given character array (buffer)
- The function does not know where this array ends
- It computes the number of chars until the first `'\0'`

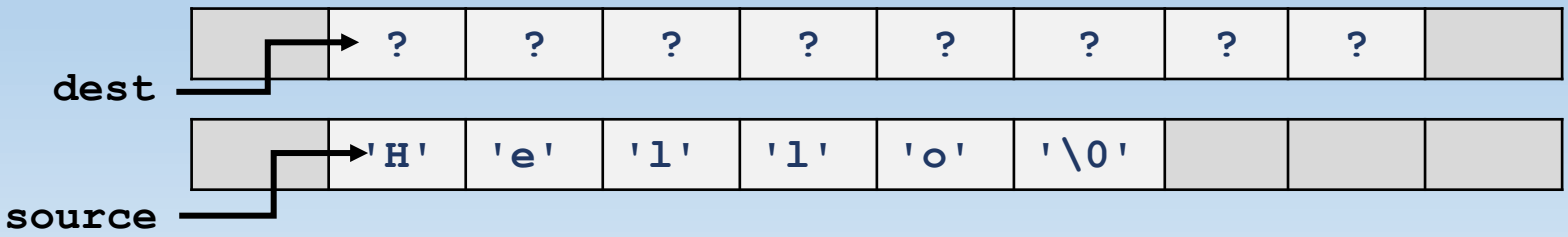


Strings functions

A function for copying a string into a given array (buffer):

```
char* my_strcpy(char* dest, const char* source) {
   int i=0;
   while(source[i]) {
      dest[i] = source[i];
      i++;
   }
   dest[i] = '\0';
   return dest;
}
```

- The calling function provides two arrays, one containing a string (**source**) and the other one (**dest**) is a buffer with sufficient space.

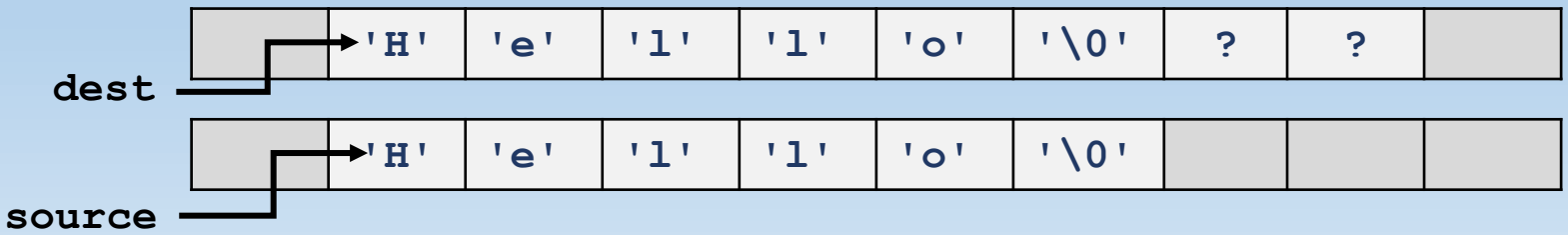


Strings functions

A function for copying a string into a given array (buffer):

```
char* my_strcpy(char* dest, const char* source) {
   int i=0;
   while(source[i]) {
      dest[i] = source[i];
      i++;
   }
   dest[i] = '\0';
   return dest;
}
```

- The calling function provides two arrays, one containing a string (**source**) and the other one (**dest**) is a buffer with sufficient space.
- The source string is copied to the destination buffer.



Strings functions

A function for copying a string into a given array (buffer):

```
char* my_strcpy(char* dest, const char* source) {
   int i=0;
   while(source[i]) {
      dest[i] = source[i];
      i++;
   }
   dest[i] = '\0';
   return dest;
}
```

- The calling function provides two arrays, one containing a string (**`source`**) and the other one (**`dest`**) is a buffer with sufficient space.
- The source string is copied to the destination buffer.
- The function returns a pointer to the destination string.
- If the destination buffer doesn't contains enough cells, it may overwrite memory cells that belong to other variables.

Strings functions

A function for copying a string into a given array (buffer):

```
char* my_strcpy(char* dest, const char* source) {
   char* dest_start = dest;
   while(*source) {
      *dest++ = *source++;
   }
   *dest = '\0';
   return dest_start;
}
```

- An alternative implementation of the same function using pointer arithmetic instead of array indexing
  - Note that `*p++` is the same as `*(p++)`, meaning that it de-references the address in `p` before incrementation.

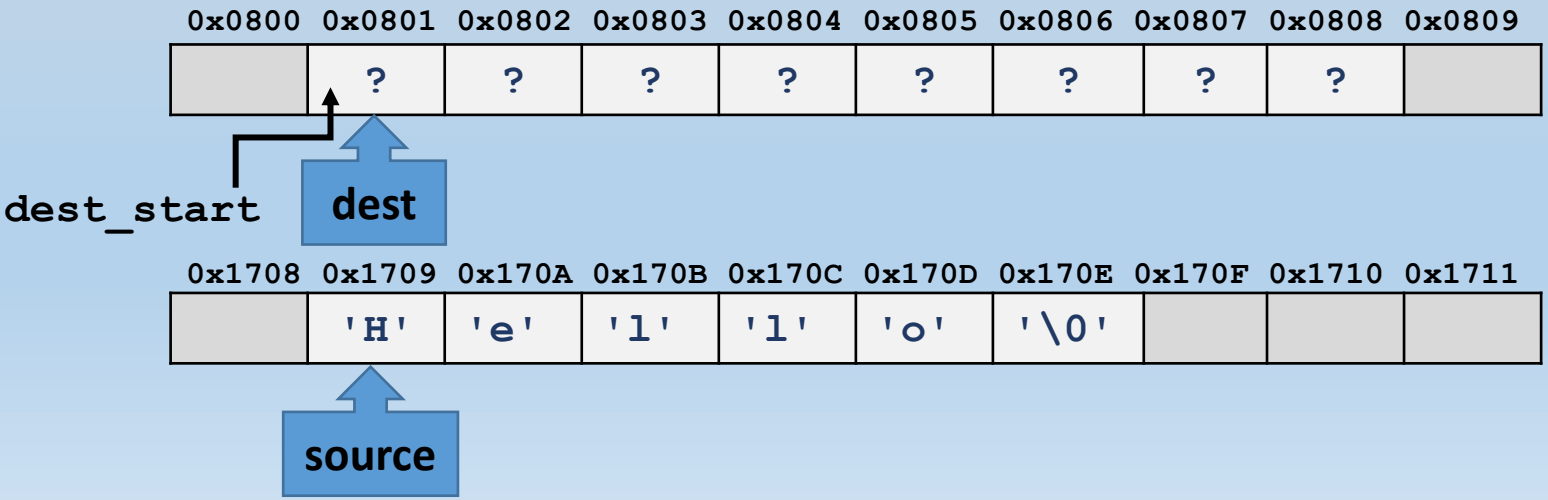
Strings functions

A function for copying a string into a given array (buffer):

```
char* my_strcpy(char* dest, const char* source) {
   char* dest_start = dest;
   while(*source) {
      *dest++ = *source++;
   }
   *dest = '\0';
   return dest_start;
}
```

Same as:

```
*dest = *source;
dest++;
source++;
```



Strings functions

- Standard library `string.h` contains many functions that work with strings:
  - `strlen()` – string length
  - `strcpy()` – copies string
  - `strncpy()` – copies string, up to given num chars
  - `strcat()` – concatenates string ← class exercise
  - `strncat()` – concatenates string, up to given num chars
  - `strcmp()` – lexicographic comparison of strings (returns -1, 0, or 1)
  - `strchr()` – returns pointer to first occurrence of given char in string
- Many more…
- See http://www.cplusplus.com/reference/cstring/

Recitation overview

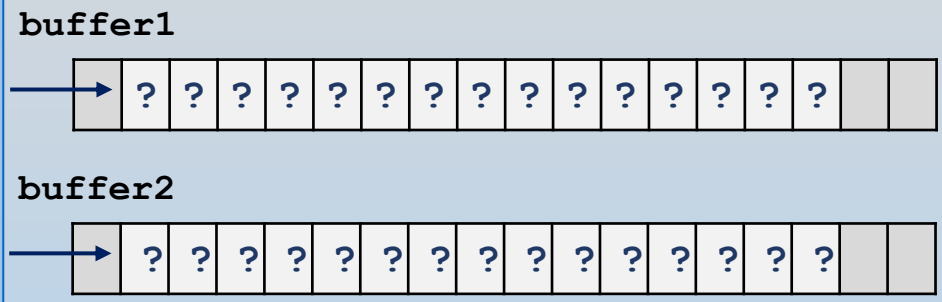
- Strings – review
  - strlen, strcpy
- Step-by-step execution example of string functions
- Class exercise – strcat

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
  strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



- local arrays are not initialized

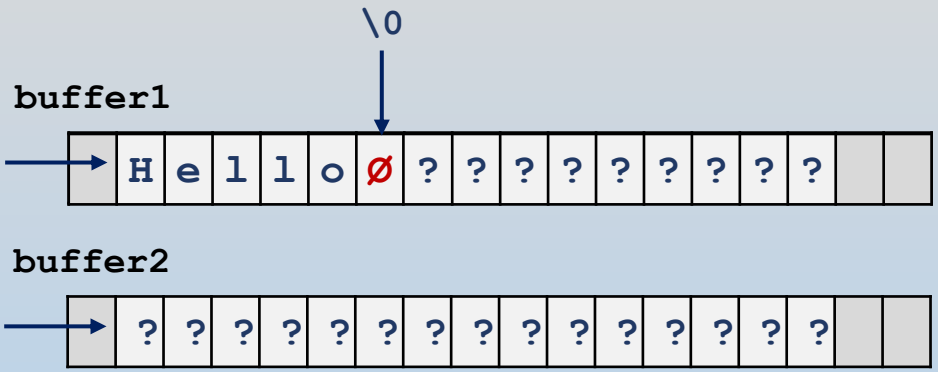
see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
  strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



String = Hello

- The buffer can be longer than the actual string.
- It has to be long enough to contain all characters + terminating `'\0'`

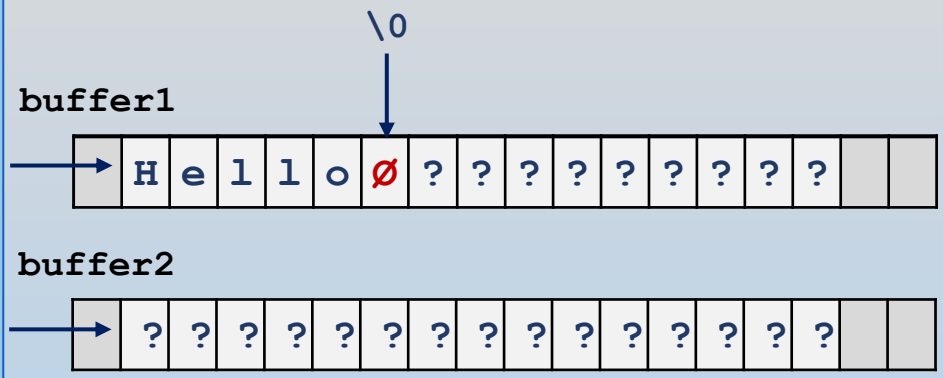
see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
→ strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



- strcat concatenates strings

see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
→ strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```

String = Hello World

- strcat concatenates strings

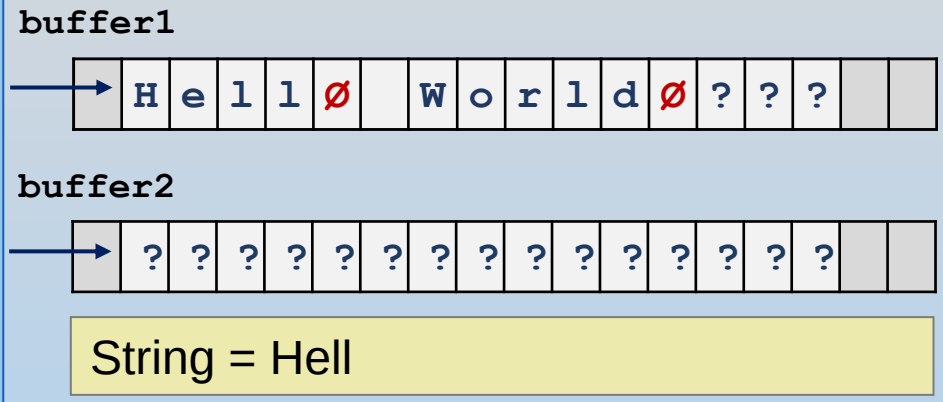
see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
  strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
→ buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



- String can be shortened simply by assigning `'\0'` to the appropriate cell.

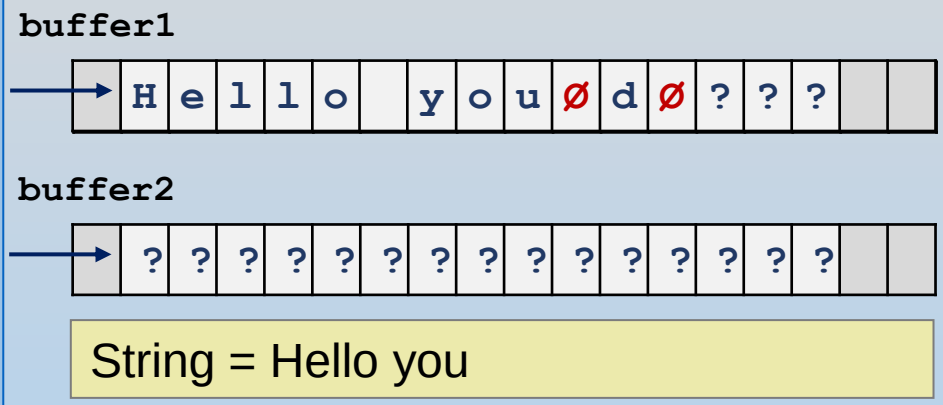
see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
  strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
→ strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



String = Hello you

- When accessing a string, the program does not see beyond the first `'\0'`

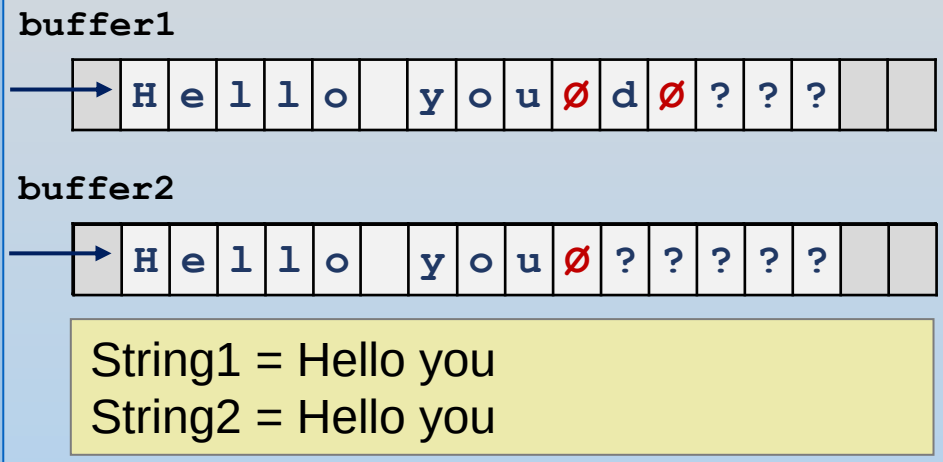
see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
  strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
→ strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



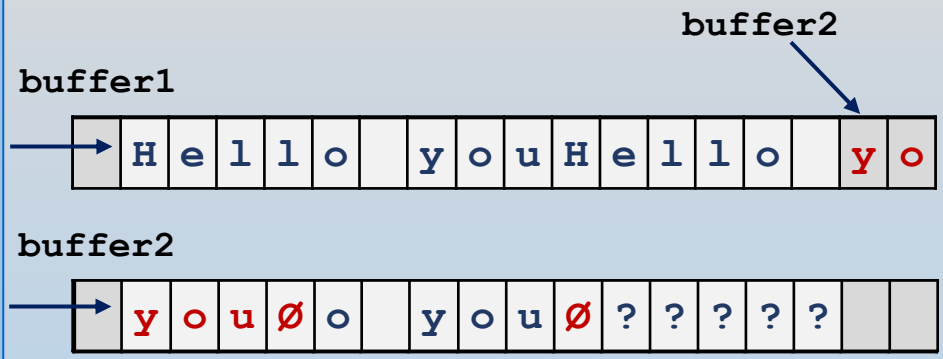
see /share/recitations/recitation08/

Strings – example

```
#include <stdio.h>
#include <string.h>
#define ARRSIZE 15

int main() {
  char buffer1[ARRSIZE],
       buffer2[ARRSIZE];

  strcpy(buffer1, "Hello");
  printf("String = %s\n",buffer1);
  strcat(buffer1, " World");
  printf("String = %s\n",buffer1);
  buffer1[4] = '\0';
  printf("String = %s\n",buffer1);
  strcat(buffer1,"o you");
  printf("String = %s\n",buffer1);
  strcpy(buffer2,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
→ strcat(buffer1,buffer1);
  printf("String1 = %s\n" , buffer1);
  printf("String2 = %s\n", buffer2);
  return 0;
}
```



String1 = Hello youHello you
String2 = you

- Concatenation overflow from buffer1 to buffer2
- **IMPLEMENTATION-DEPENDENT!**

see /share/recitations/recitation08/

Recitation overview

- Strings – review
  - strlen, strcpy
- Step-by-step execution example of string functions
- Class exercise – strcat

Class assignment

A function for concatenating two strings:

```
char* my_strcat(char* dest, const char* source) {



   /*** WRITE CODE HERE ***/



   return dest;  // return pointer to dest string
}
```

- It assumes that the destination buffer contains enough cells.
- Copy code file `strings.c` from `/share/recitations/recitation08/` and complete function my_strcat()
- Code should compile without errors/warnings and run:

```
sentence before: hello
sentence after:  hello world
```

Class assignment – solution

A function for concatenating two strings:

```
char* my_strcat(char* dest, const char* source) {
   int i_s=0, i_d=0;
   while(dest[i_d]) i_d++;
   while(source[i_s]) {
      dest[i_d] = source[i_s];
      i_s++;
      i_d++;
   }
   dest[i_d] = '\0';
   return dest;  // return pointer to dest string
}
```

- It assumes that the destination buffer contains enough cells.

Class assignment – solution

A function for concatenating two strings:

```
char* my_strcat(char* dest, const char* source) {
  int length_dest=my_strlen(dest);
  my_strcpy(dest+length_dest, source);
  return dest;  // return pointer to dest string
}
```

- Alternative implementation using my_strlen() and my_strcpy().
- **`dest+length_dest`** is the address of the terminating **`'\0'`**
- Function copies **`source`** into buffer starting from this point.